

# Introduction to CLCD

Sep 29<sup>th</sup> 2026

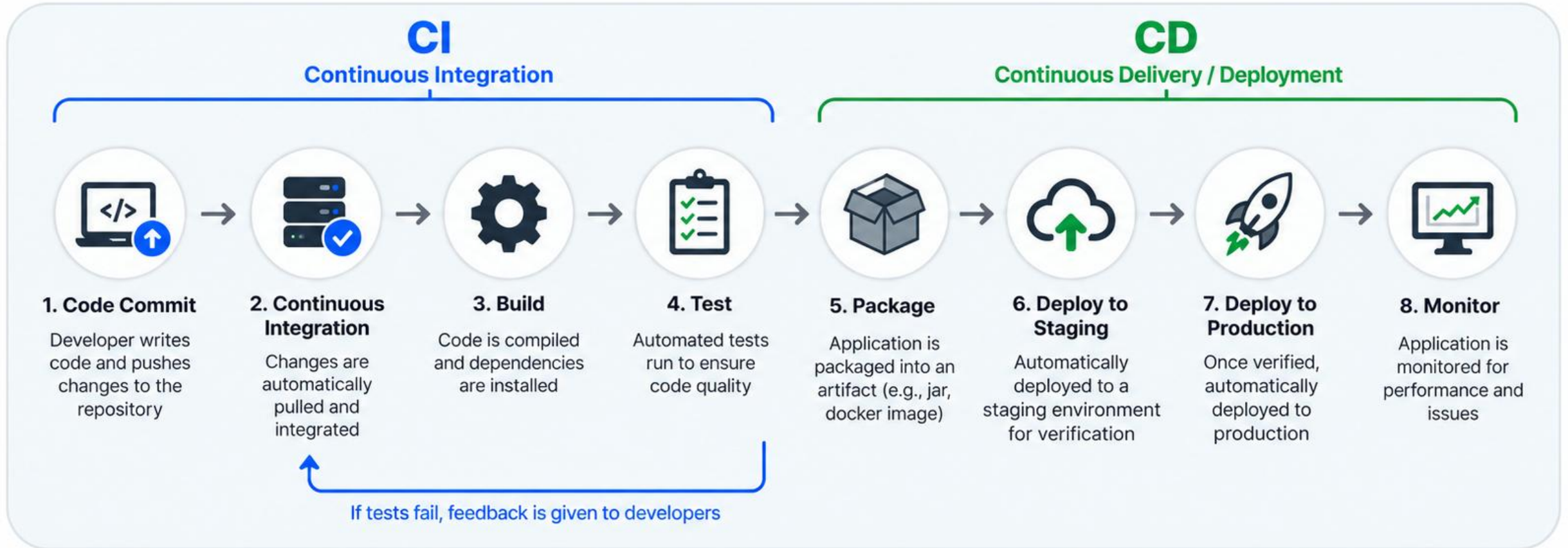
# What is CI/CD?

## **Definition: Continuous Integration / Continuous Delivery (or Deployment)**

A set of practices that **automate** building, testing, and releasing software whenever developers make changes.

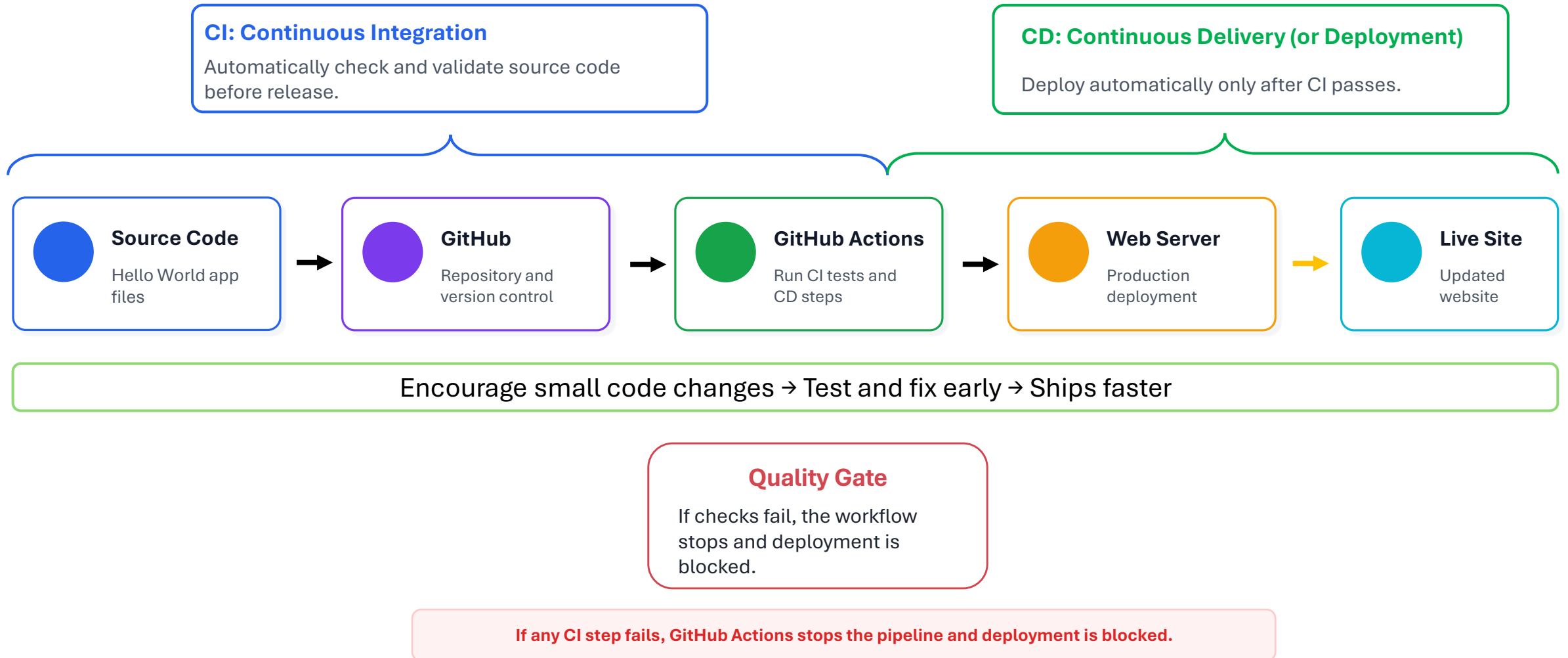
**Flow:** Code changes from commit to production through an automated pipeline.

# Typical CI/CD



# CI/CD End-to-End Pipeline Flow

Automate the path from source code to production in practice.



# What is GitHub Actions?

CI/CD *YAML* workflow and its building blocks.

```
1  # .github/workflows/cicd.yml 1
2  name: Hello World CI/CD
3
4  on: 2
5    pull_request:
6      branches:
7        - main
8    push:
9      branches:
10       - main
11
12  jobs: 3
13    test:
14      runs-on: ubuntu-latest 5
15      4 steps:
16        - name: Checkout Source Code
17          uses: actions/checkout@v7 6
18        - name: Test 5 + 4
19          run: |
20            RESULT=$((5 + 4))
21        - name: Validate Source Code
22          run: npm run test:html
23
24    deploy:
25      needs:
26        - test
27      runs-on: ubuntu-latest
28      steps:
29        - name: Deploy to Cloudflare
30          uses: cloudflare/wrangler-action@v4
```

## 1 Workflow

The full automation pipeline defined in a YAML file.

```
# .github/workflows/cicd.yml
name: Hello World CI/CD
```

## 2 Trigger

The event that starts the workflow, such as push or pull request.

```
on:
  pull_request:
  push:
```

## 3 Job

A group of related tasks, such as test or deploy.

```
jobs:
  test:
  deploy:
```

## 4 Step

One command or reusable action inside a job.

```
steps:
  - name: Checkout Source Code
  - name: Validate Source Code
```

## 5 Runner

The machine that executes the workflow.

```
runs-on: ubuntu-latest
```

## 6 Action

A reusable automation component called from a step.

```
uses: actions/checkout@v7
uses: cloudflare/wrangler-
action@v4
```

# What does a typical CI checks?

The quality gate before deployment.

## CI tests inside GitHub Actions

**1 Check source file exists**  
`public/index.html`

**2 Quality Test**  
`5 + 4 = 9`

**3 Validate source code**  
HTML Lint Rules

**4 Security Scanning**  
Secrets Scanning

### GitHub Actions log

Checking `public/index.html...`

✓ Required source file exists

Testing: `5 + 4 = 9`

✓ Calculation test passed

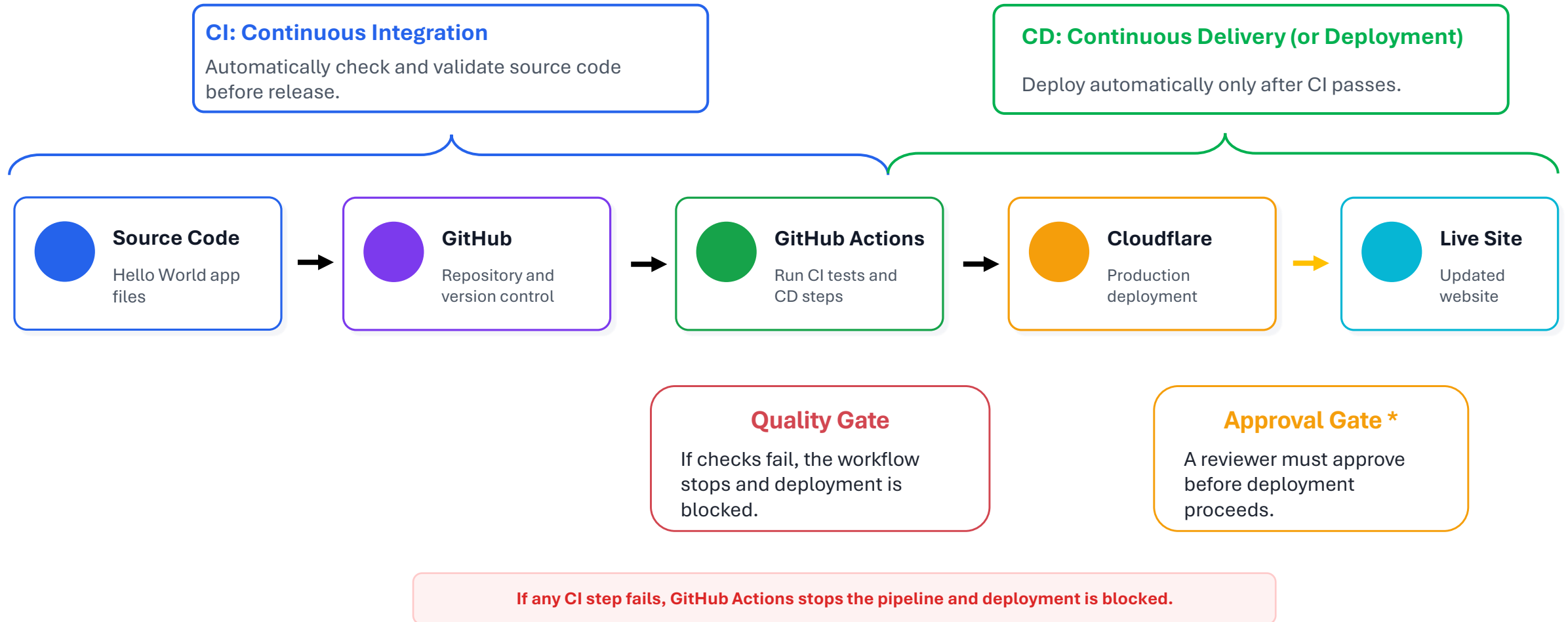
Scanned 1 files, no errors found

✓ ALL CI TESTS PASSED

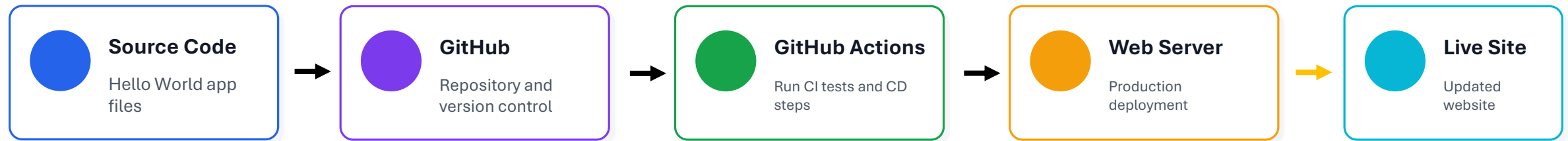
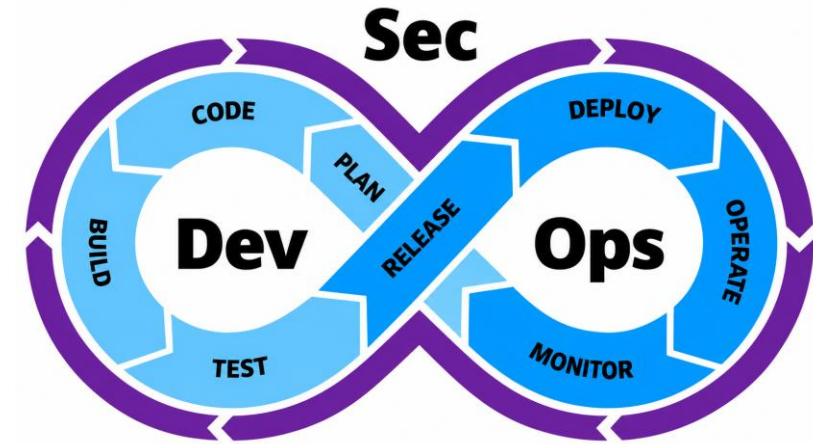
**Result: failed CI blocks deployment; passed CI allows deployment**

# Demo: CI/CD End-to-End Pipeline Flow

Automate the path from source code to production in practice.



# CI/CD Pipeline (DevSecOps)



## Security Considerations / Guard Rails

### Quality Gate

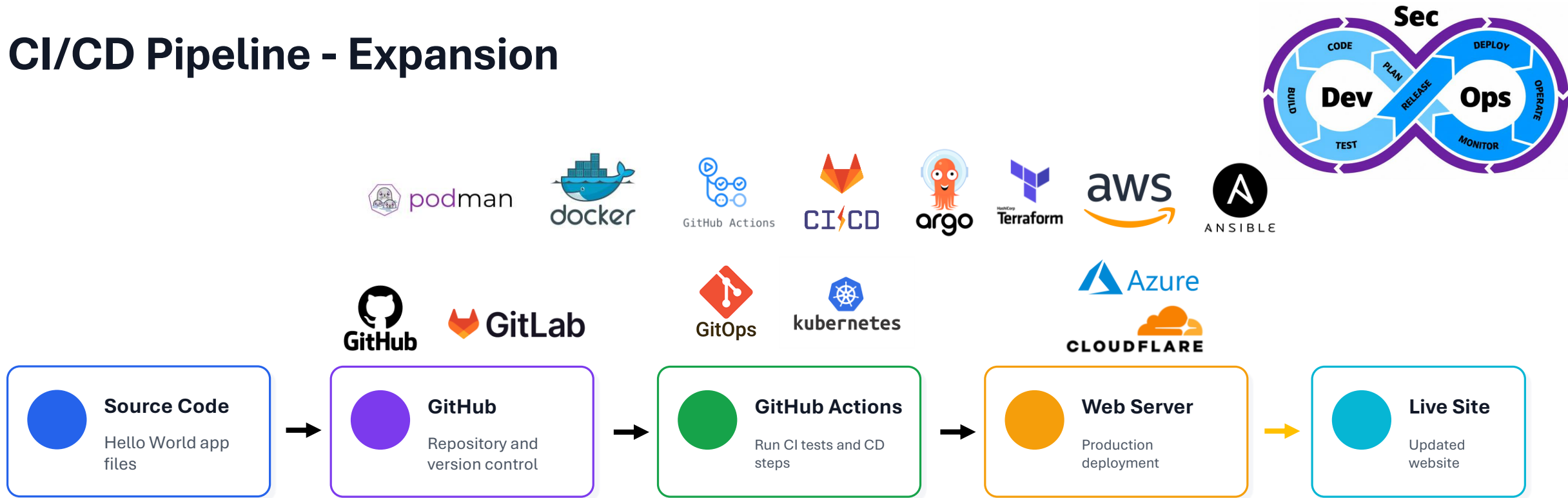
If checks fail, the workflow stops and deployment is blocked.

### Approval Gate \*

A reviewer must approve before deployment proceeds.

If any CI step fails, GitHub Actions stops the pipeline and deployment is blocked.

# CI/CD Pipeline - Expansion



## Security Considerations / Guard Rails

SAST — Static Application Security Testing → SonarQube / Snyc

SCA — Software Composition Analysis → Snyc / Trivy/ Dependabot

Secrets — Secret Scanning / Secrets Detection → Trivy / GitHub Secret Scanning / GitLeaks

IaC — Infrastructure as Code Security Scanning → Checkov / Trivy

Containers — Container Security / Container Image Scanning → Trivy / JFrog

DAST — Dynamic Application Security Testing → OWASP ZAP / Burp Suite



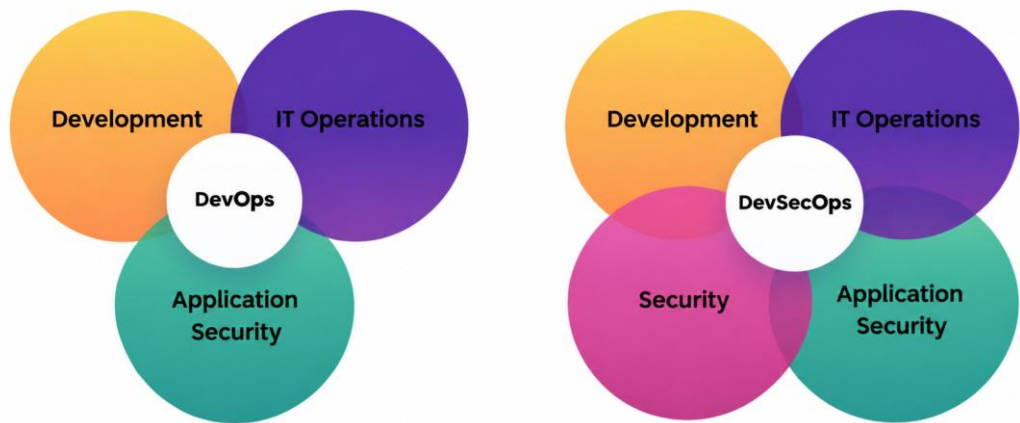
## Dashboard and Management

CSPM — Cloud Security Posture Management

SIEM — Security information and Event Management

Logging — ELK Stack / Open Search

DevOps VS DevSecOps



| Component            | DevOps                                                                           | DevSecOps                                                                               |
|----------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Development          | Develop and maintain applications                                                | Develop applications with security integrated into engineering workflows                |
| IT Operations        | Deploy, operate and maintain applications                                        | Deploy and operate applications with security incorporated into operational processes   |
| Application Security | Application security is recognized, but may <b>remain a separate function</b>    | <b>Application security is integrated</b> into the delivery process                     |
| Security             | Security activities may involve separate teams or reviews later in the lifecycle | <b>Security responsibilities are shared</b> across development, security and operations |
| Primary emphasis     | Collaboration, automation and faster delivery                                    | Collaboration, automation and secure software delivery                                  |

# Why do we need CI/CD?

| Problem without it           | What CI/CD does                               |
|------------------------------|-----------------------------------------------|
| Bugs found late              | Tests run on every commit                     |
| "Works on my machine"        | Builds run in a clean, consistent environment |
| Manual, error-prone releases | Deploys are scripted and repeatable           |
| Slow releases (weeks)        | Ship in minutes or hours                      |
| Big, risky merges            | Small, frequent changes                       |
| Hard rollbacks               | Automated rollback and canary rollouts        |
| No audit trail               | Every change is logged and traceable          |
| Security checks skipped      | Scans run automatically in the pipeline       |